Trees

Chapter 11

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Introduction to Trees
- Applications of Trees (*not currently included in overheads*)
- Tree Traversal
- Spanning Trees
- Minimum Spanning Trees (*not currently included in overheads*)

Introduction to Trees

Section 11.1

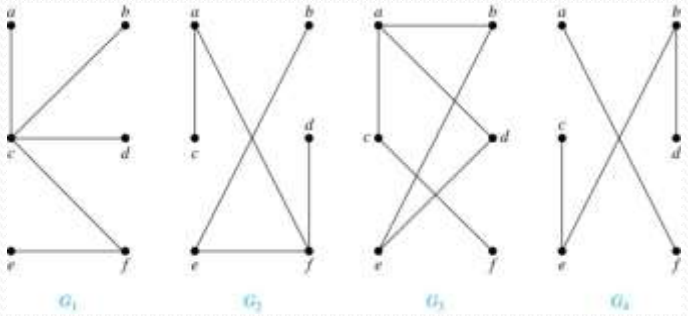
Section Summary

- Introduction to Trees
- Rooted Trees
- Trees as Models
- Properties of Trees

Trees

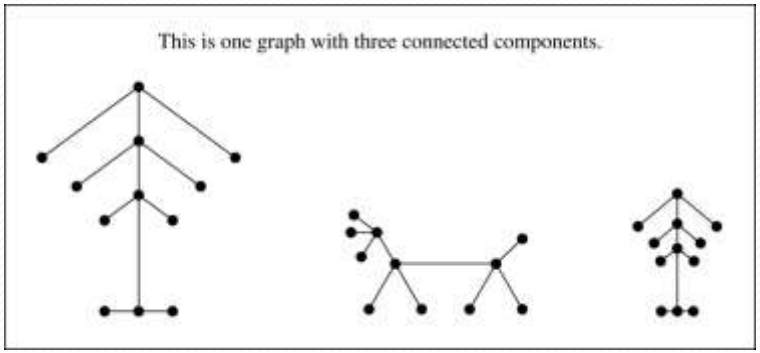
Definition: A *tree* is a connected undirected graph with no simple circuits.

Example: Which of these graphs are trees?



Solution: G_1 and G_2 are trees - both are connected and have no simple circuits. Because e, b, a, d, e is a simple circuit, G_3 is not a tree. G_4 is not a tree because it is not connected.

Definition: A *forest* is a graph that has no simple circuit, but is not connected. Each of the connected components in a forest is a tree.



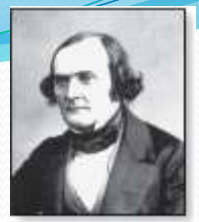
Trees (*continued*)

Theorem: An undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.

Proof: Assume that T is a tree. Then T is connected with no simple circuits. Hence, if x and y are distinct vertices of T, there is a simple path between them (by Theorem 1 of Section 10.4). This path must be unique - for if there were a second path, there would be a simple circuit in T (by Exercise 59 of Section 10.4). Hence, there is a unique simple path between any two vertices of a tree.

Now assume that there is a unique simple path between any two vertices of a graph T. Then T is connected because there is a path between any two of its vertices. Furthermore, T can have no simple circuits since if there were a simple circuit, there would be two paths between some two vertices.

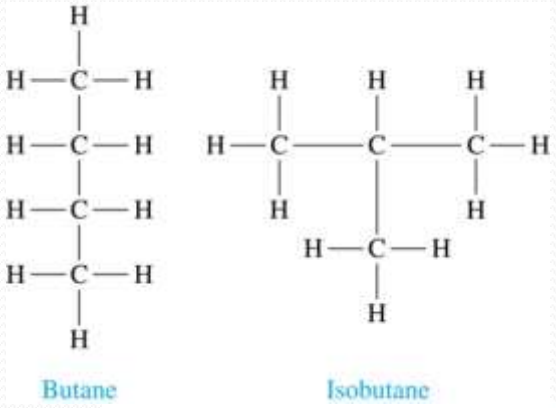
Hence, a graph with a unique simple path between any two vertices is a tree. ◀

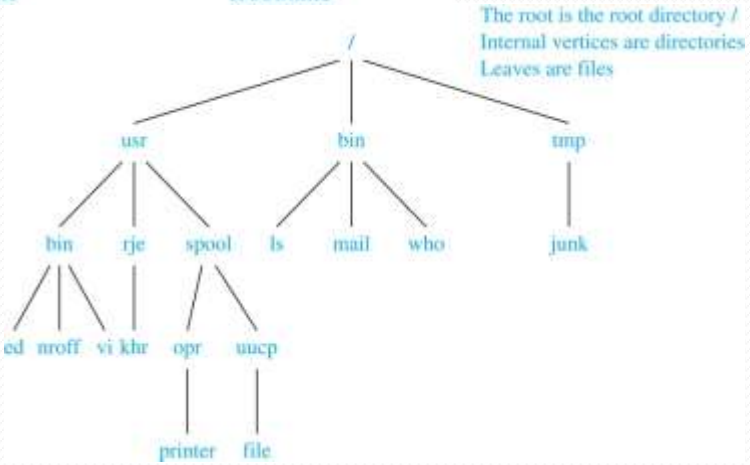


Arthur Cayley
(1821-1895)

Trees as Models

- Trees are used as models in computer science, chemistry, geology, botany, psychology, and many other areas.
- Trees were introduced by the mathematician Cayley in 1857 in his work counting the number of isomers of saturated hydrocarbons. The two isomers of butane are shown at the right.
- The organization of a computer file system into directories, subdirectories, and files is naturally represented as a tree.
- Trees are used to represent the structure of organizations.



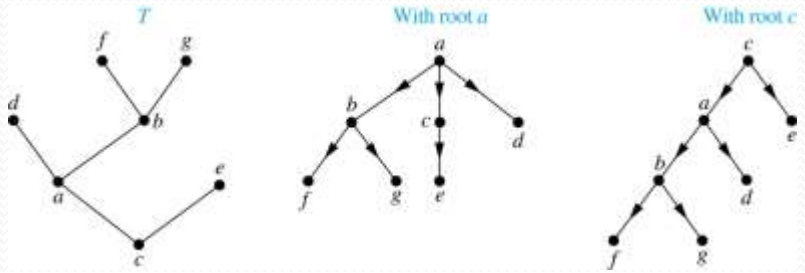




Rooted Trees

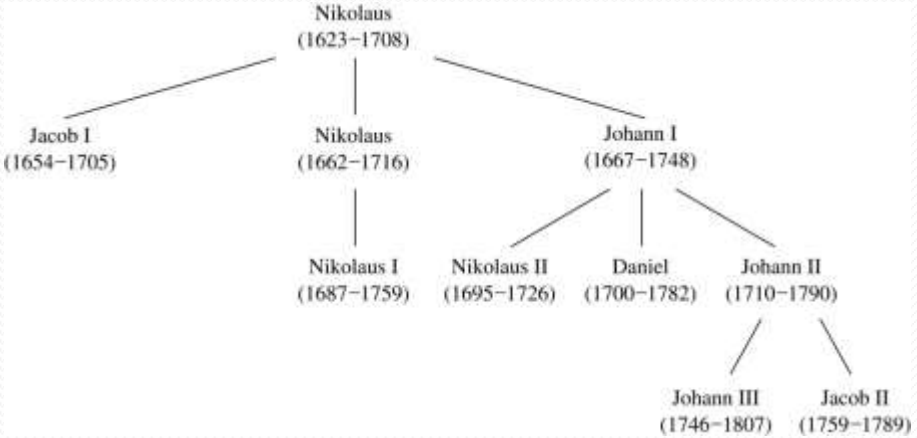
Definition: A *rooted tree* is a tree in which one vertex has been designated as the *root* and every edge is directed away from the root.

An unrooted tree is converted into different rooted trees when different vertices are chosen as the root.



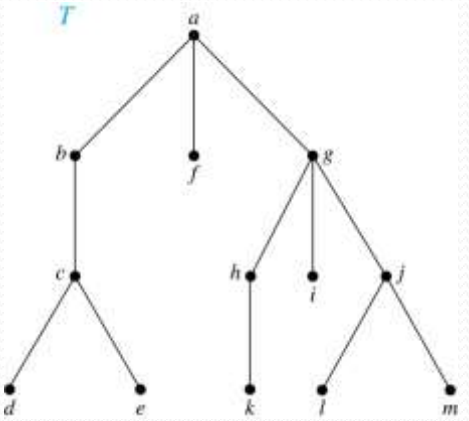
Rooted Tree Terminology

- Terminology for rooted trees is a mix from botany and genealogy (such as this family tree of the Bernoulli family of mathematicians).



- If v is a vertex of a rooted tree other than the root, the ***parent*** of v is the unique vertex u such that there is a directed edge from u to v. When u is a parent of v, v is called a ***child*** of u. Vertices with the same parent are called ***siblings***.
- The ***ancestors*** of a vertex are the vertices in the path from the root to this vertex, excluding the vertex itself and including the root. The ***descendants*** of a vertex v are those vertices that have v as an ancestor.
- A vertex of a rooted tree with no children is called a ***leaf***. Vertices that have children are called ***internal*** *vertices*.
- If a is a vertex in a tree, the ***subtree*** with a as its root is the subgraph of the tree consisting of a and its descendants and all edges incident to these descendants.

Terminology for Rooted Trees



Example: In the rooted tree T (with root a):

(i) Find the parent of c, the children of g, the siblings of h, the ancestors of e, and the descendants of b.

(ii) Find all internal vertices and all leaves.

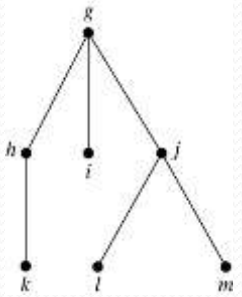
(iii) What is the subtree rooted at g?

Solution:

(i) The parent of c is b. The children of g are h, i, and j. The siblings of h are i and j. The ancestors of e are c, b, and a. The descendants of b are c, d, and e.

(ii) The internal vertices are a, b, c, g, h, and j. The leaves are d, e, f, i, k, l, and m.

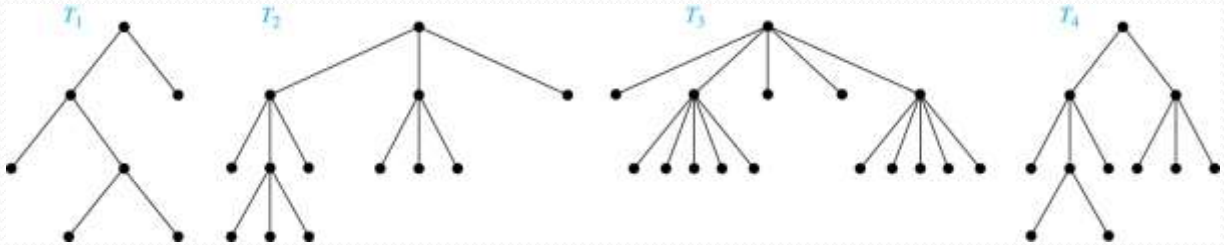
(iii) We display the subtree rooted at g.



m-ary Rooted Trees

Definition: A rooted tree is called an ***m-ary tree*** if every internal vertex has no more than *m* children. The tree is called a ***full m-ary tree*** if every internal vertex has exactly *m* children. An *m*-ary tree with $m = 2$ is called a *binary* tree.

Example: Are the following rooted trees full *m*-ary trees for some positive integer *m*?



Solution: T_1 is a full binary tree because each of its internal vertices has two children. T_2 is a full 3-ary tree because each of its internal vertices has three children. In T_3 each internal vertex has five children, so T_3 is a full 5-ary tree. T_4 is not a full *m*-ary tree for any m because some of its internal vertices have two children and others have three children.

Properties of Trees

Theorem 2: A tree with n vertices has $n - 1$ edges.

***Proof* (*by mathematical induction*):**

BASIS STEP: When $n = 1$, a tree with one vertex has no edges. Hence, the theorem holds when $n = 1$.

INDUCTIVE STEP: Assume that every tree with k vertices has $k - 1$ edges.

Suppose that a tree T has $k + 1$ vertices and that v is a leaf of T. Let w be the parent of v. Removing the vertex v and the edge connecting w to v produces a tree T' with k vertices. By the inductive hypothesis, T' has $k - 1$ edges. Because T has one more edge than T', we see that T has k edges. This completes the inductive step. ◀

Counting Vertices in Full *m*-Ary Trees

Theorem 3: A full *m*-ary tree with *i* internal vertices has $n = mi + 1$ vertices.

Proof : Every vertex, except the root, is the child of an internal vertex. Because each of the *i* internal vertices has *m* children, there are *mi* vertices in the tree other than the root. Hence, the tree contains $n = mi + 1$ vertices. ◄

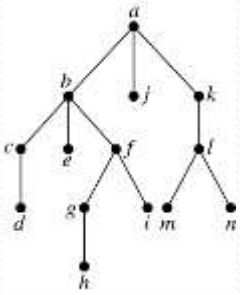
Level of vertices and height of trees

- When working with trees, we often want to have rooted trees where the subtrees at each vertex contain paths of approximately the same length.
- To make this idea precise we need some definitions:
  - The ***level*** of a vertex *v* in a rooted tree is the length of the unique path from the root to this vertex.
  - The ***height*** of a rooted tree is the maximum of the levels of the vertices.

Example:

(*i*) Find the level of each vertex in the tree to the right.

(*ii*) What is the height of the tree?



Solution:

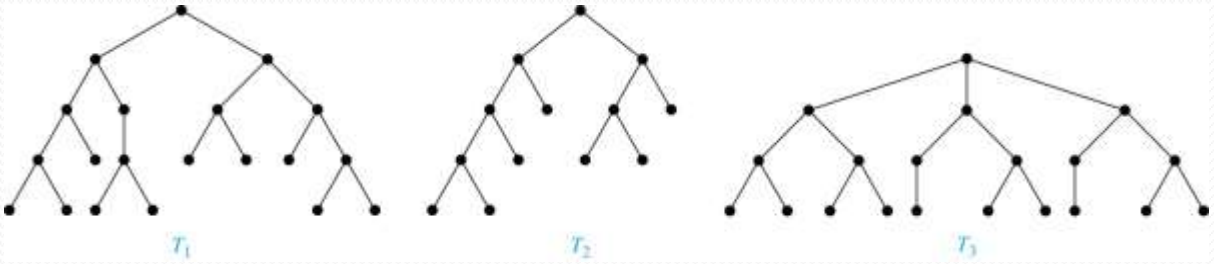
(*i*) The root *a* is at level 0. Vertices *b*, *j*, and *k* are at level 1. Vertices *c*, *e*, *f*, and *l* are at level 2. Vertices *d*, *g*, *i*, *m*, and *n* are at level 3. Vertex *h* is at level 4.

(*ii*) The height is 4, since 4 is the largest level of any vertex.

Balanced *m*-Ary Trees

Definition: A rooted *m*-ary tree of height *h* is *balanced* if all leaves are at levels *h* or $h - 1$.

Example: Which of the rooted trees shown below is balanced?



Solution: T_1 and T_3 are balanced, but T_2 is not because it has leaves at levels 2, 3, and 4.

Tree Traversal

Section 11.3

Section Summary

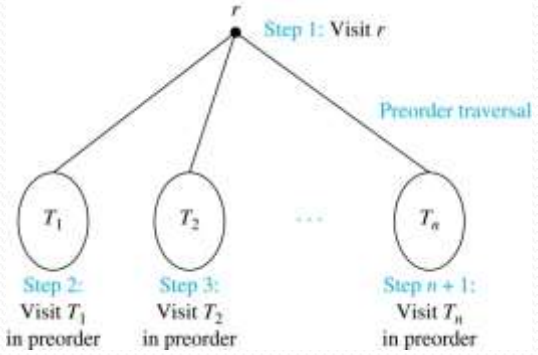
- Universal Address Systems (*not currently included in overheads*)
- Traversal Algorithms
- Infix, Prefix, and Postfix Notation

Tree Traversal

- Procedures for systematically visiting every vertex of an ordered tree are called *traversals*.
- The three most commonly used *traversals* are *preorder traversal*, *inorder traversal*, and *postorder traversal*.

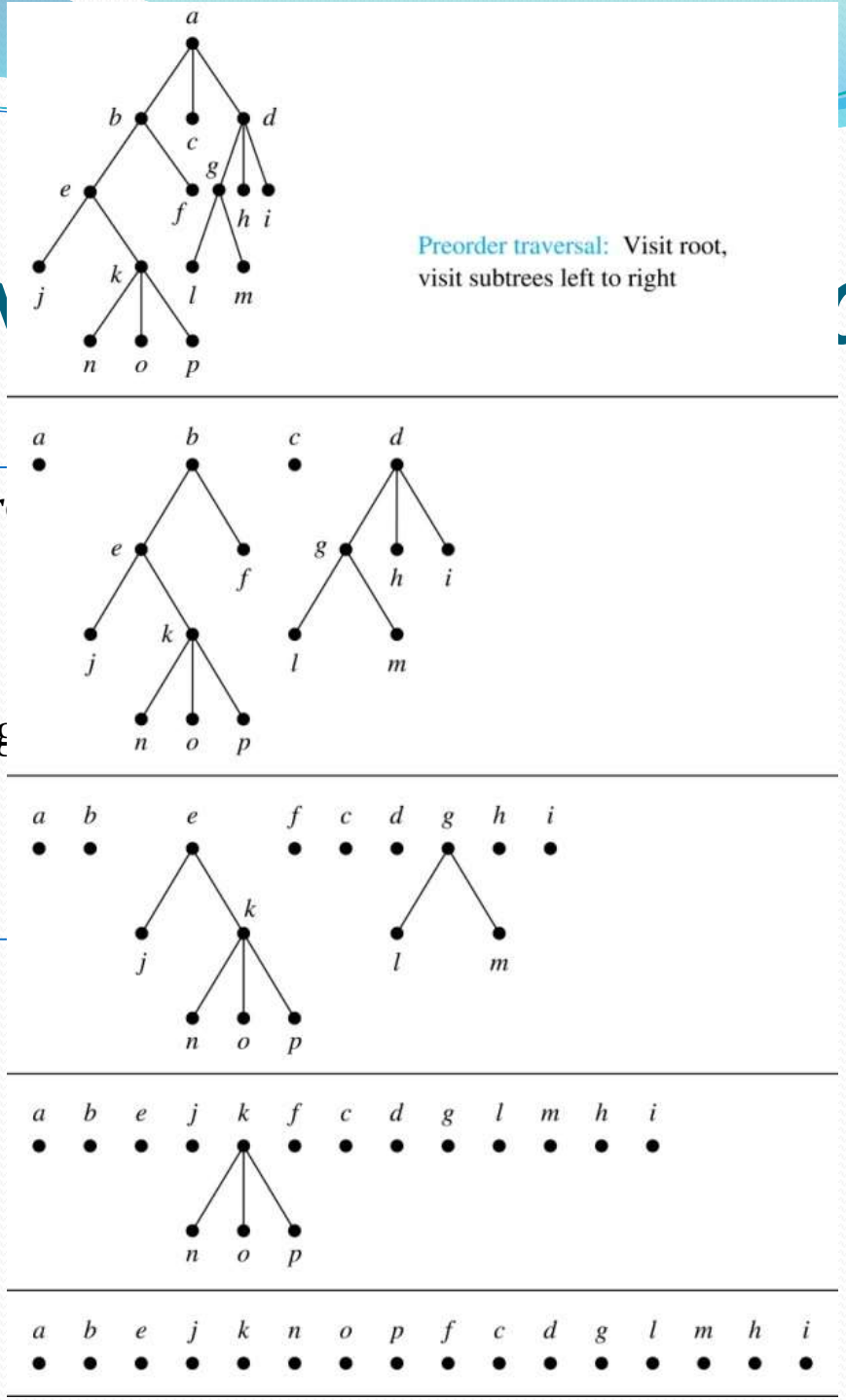
Preorder Traversal

Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *preorder traversal* of T. Otherwise, suppose that T_1, T_2, …, T_n are the subtrees of r from left to right in T. The preorder traversal begins by visiting r, and continues by traversing T_1 in preorder, then T_2 in preorder, and so on, until T_n is traversed in preorder.



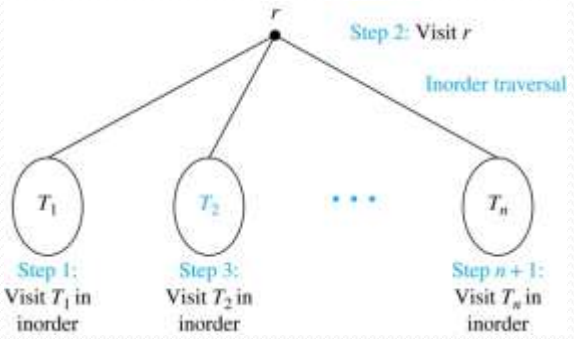
Preorder Trav d)

procedure *preorder* (*T*: ordered r
tree)
r := root of *T*
list *r*
for each child *c* of *r* from left to rig
  T(*c*) := subtree with *c* as root
  preorder(*T*(*c*))



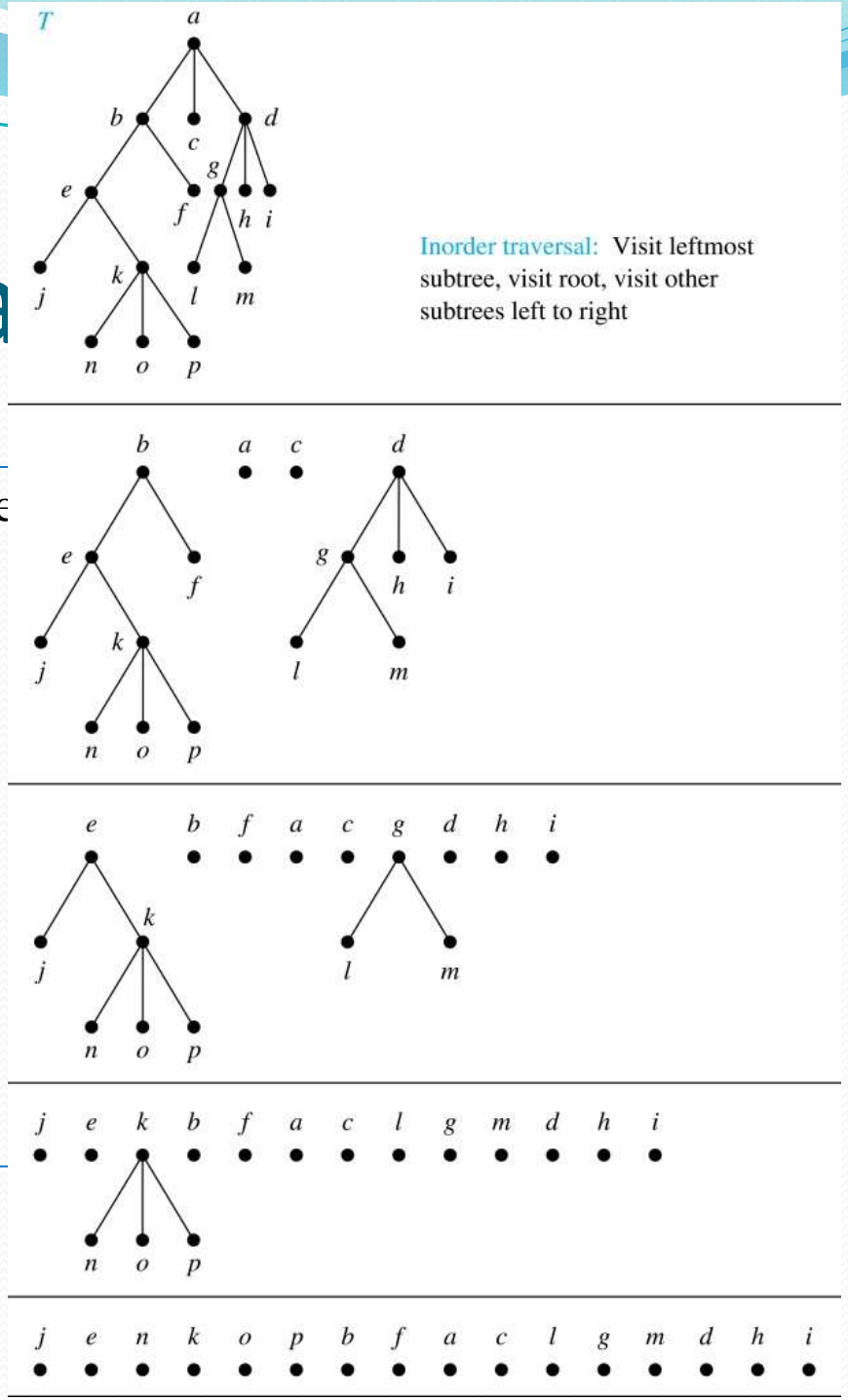
Inorder Traversal

Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *inorder traversal* of T. Otherwise, suppose that T_1, T_2, …, T_n are the subtrees of r from left to right in T. The inorder traversal begins by traversing T_1 in inorder, then visiting r, and continues by traversing T_2 in inorder, and so on, until T_n is traversed in inorder.



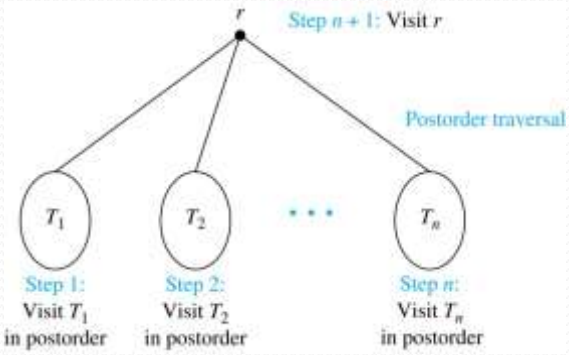
Inorder Traversa

```
procedure inorder (T: ordered rooted tree
r := root of T
if r is a leaf then list r
else
    l := first child of r from left to right
    T(l) := subtree with l as its root
    inorder(T(l))
    list(r)
    for each child c of r from left to right
        T(c) := subtree with c as root
        inorder(T(c))
```



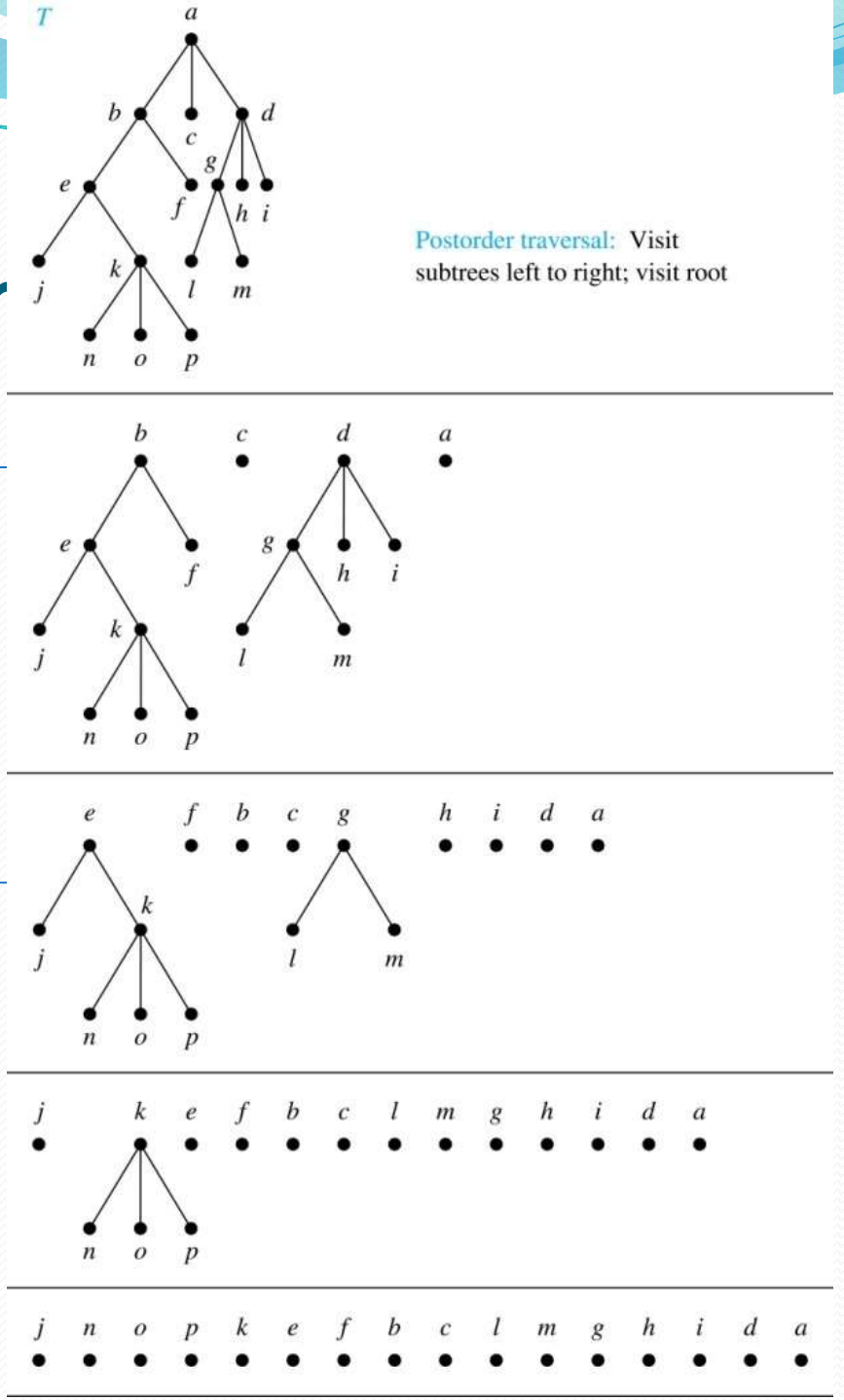
Postorder Traversal

Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *postorder traversal* of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees of r from left to right in T. The postorder traversal begins by traversing T_1 in postorder, then T_2 in postorder, and so on, after T_n is traversed in postorder, r is visited.



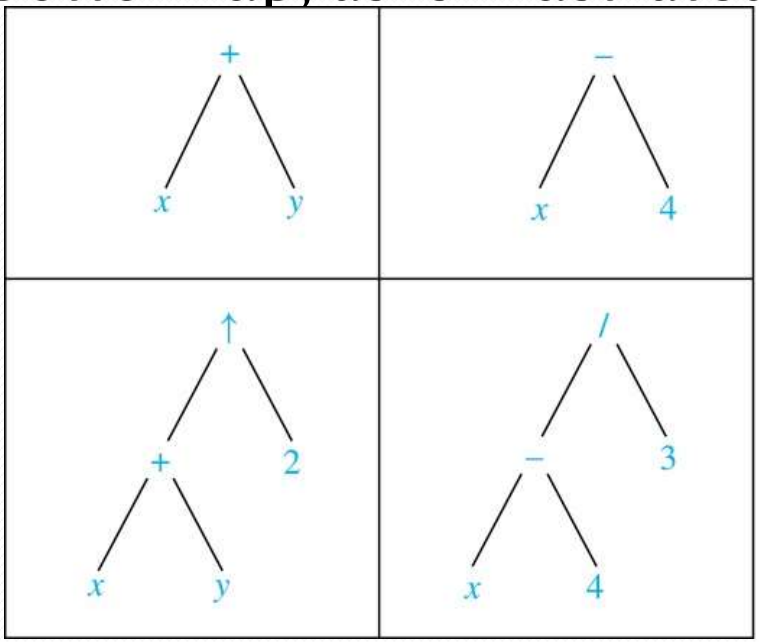
Postorder Traver

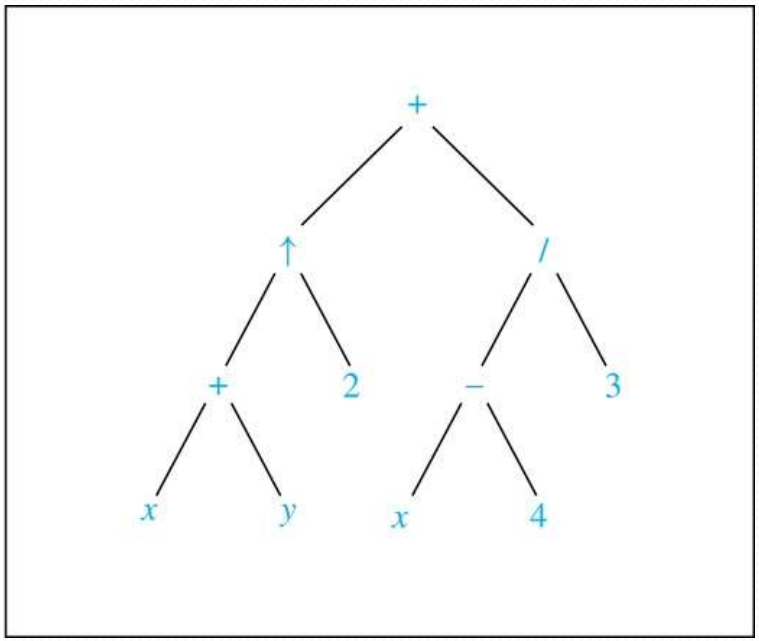
procedure *postordered* (*T*: ordered rooted
r := root of *T*
for each child *c* of *r* from left to right
    T(*c*) := subtree with *c* as root
    postorder(*T*(*c*))
list *r*



Expression Trees

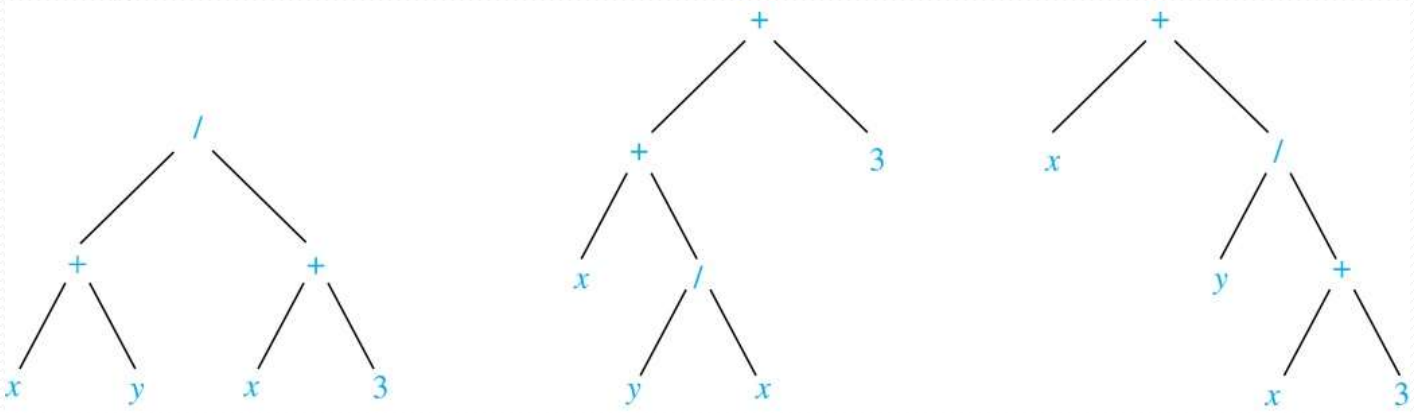
- Complex expressions can be represented using ordered rooted trees.
- Consider the expression $((x + y) \uparrow 2) + ((x - 4)/3)$.
- A binary tree for the expression can be built from the bottom up, as is illustrated here.





Infix Notation

- An inorder traversal of the tree representing an expression produces the original expression when parentheses are included except for unary operations, which now immediately follow their operands.
- We illustrate why parentheses are needed with an example that displays three trees all yield the same infix representation.



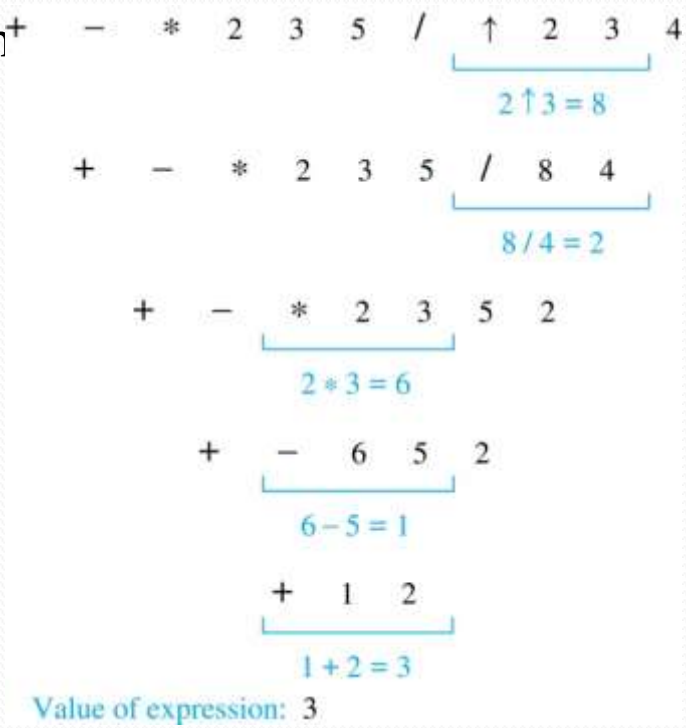


Jan Łukasiewicz
(1878-1956)

Prefix Notation

- When we traverse the rooted tree representation of an expression in preorder, we obtain the *prefix* form of the expression. Expressions in prefix form are said to be in *Polish notation*, named after the Polish logician Jan Łukasiewicz.
- Operators precede their operands in the prefix form of an expression. Parentheses are not needed as the representation is unambiguous.
- The prefix form of $((x + y) \uparrow 2) + ((x - 4)/3)$ is $+ \uparrow + x\ y\ 2\ /\ -\ x\ 4\ 3$.
- Prefix expressions are evaluated by working from right to left. When we encounter an operator, we perform the corresponding operation with the two operations to the right.

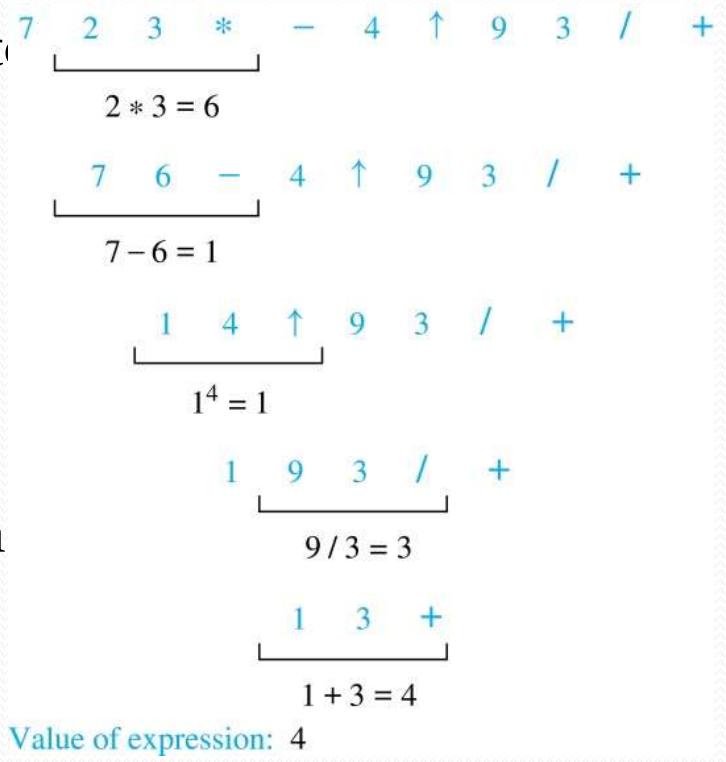
Example: We show the steps used to evaluate a particular prefix expression:



Postfix Notation

- We obtain the *postfix form* of an expression by traversing its binary trees in postorder. Expressions written in postfix form are said t be in *reverse Polish notation*.
- Parentheses are not needed as the postfix form is unambiguous.
- $x\ y + 2 \uparrow x\ 4 - 3 / +$ is the postfix form of $((x + y) \uparrow 2\ ) + ((x - 4)/3)$.
- A binary operator follows its two operands. So, to evaluate an expression one works from left to right, carrying out an operation represented by an operator on its preceding operands.

Example: We show the steps used to evaluate a particular postfix expression.



Spanning Trees

Section 11.4

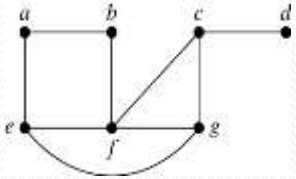
Section Summary

- Spanning Trees
- Depth-First Search
- Breadth-First Search
- Backtracking Applications (*not currently included in overheads*)
- Depth-First Search in Directed Graphs

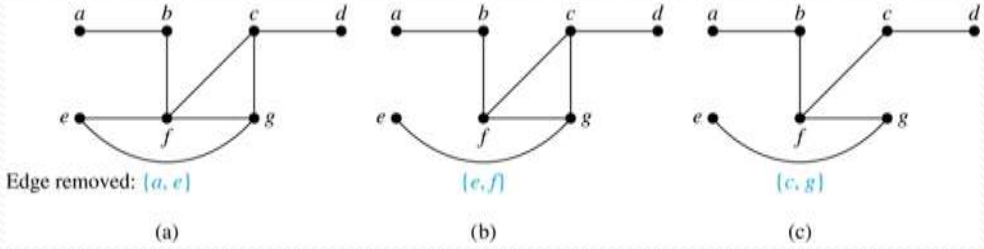
Spanning Trees

Definition: Let G be a simple graph. A spanning tree of G is a subgraph of G that is a tree containing every vertex of G.

Example: Find the spanning tree of this graph:



Solution: The graph is connected, but is not a tree because it contains simple circuits. Remove the edge $\{a, e\}$. Now one simple circuit is gone, but the remaining subgraph still has a simple circuit. Remove the edge $\{e, f\}$ and then the edge $\{c, g\}$ to produce a simple graph with no simple circuits. It is a spanning tree, because it contains every vertex of the original graph.



Spanning Trees (*continued*)

Theorem: A simple graph is connected if and only if it has a spanning tree.

Proof: Suppose that a simple graph G has a spanning tree T. T contains every vertex of G and there is a path in T between any two of its vertices. Because T is a subgraph of G, there is a path in G between any two of its vertices. Hence, G is connected.

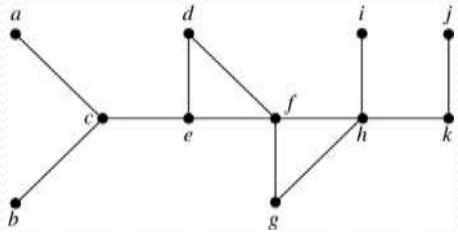
Now suppose that G is connected. If G is not a tree, it contains a simple circuit. Remove an edge from one of the simple circuits. The resulting subgraph is still connected because any vertices connected via a path containing the removed edge are still connected via a path with the remaining part of the simple circuit. Continue in this fashion until there are no more simple circuits. A tree is produced because the graph remains connected as edges are removed. The resulting tree is a spanning tree because it contains every vertex of G.

Depth-First Search

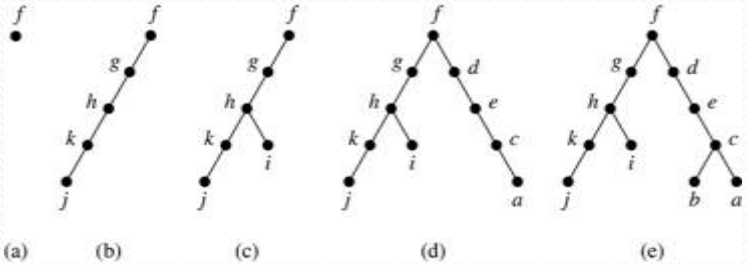
- To use *depth-first search* to build a spanning tree for a connected simple graph first arbitrarily choose a vertex of the graph as the root.
  - Form a path starting at this vertex by successively adding vertices and edges, where each new edge is incident with the last vertex in the path and a vertex not already in the path. Continue adding vertices and edges to this path as long as possible.
  - If the path goes through all vertices of the graph, the tree consisting of this path is a spanning tree.
  - Otherwise, move back to the next to the last vertex in the path, and if possible, form a new path starting at this vertex and passing through vertices not already visited. If this cannot be done, move back another vertex in the path.
  - Repeat this procedure until all vertices are included in the spanning tree.

Depth-First Search (*continued*)

Example: Use depth-first search to find a spanning tree of this graph.



Solution: We start arbitrarily with vertex *f*. We build a path by successively adding an edge that connects the last vertex added to the path and a vertex not already in the path, as long as this is possible. The result is a path that connects *f*, *g*, *h*, *k*, and *j*. Next, we return to *k*, but find no new vertices to add. So, we return to *h* and add the path with one edge that connects *h* and *i*. We next return to *f*, and add the path connecting *f*, *d*, *e*, *c*, and *a*. Finally, we return to *c* and add the path connecting *c* and *b*. We now stop because all vertices have been added.



Depth-First Search Algorithm

- We now use pseudocode to specify depth-first search. In this recursive algorithm, after adding an edge connecting a vertex v to the vertex w, we finish exploring w before we return to v to continue exploring from v.

procedure *DFS*(G: connected graph with vertices $v_1, v_2, \ldots, v_n$)
T := tree consisting only of the vertex v_1
visit(v_1)

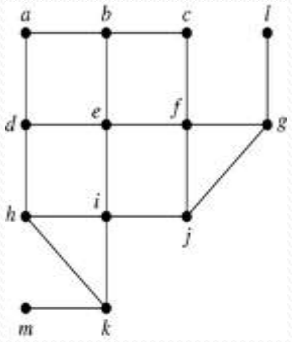
procedure *visit*(v: vertex of G)
for each vertex w adjacent to v and not yet in T
  add vertex w and edge $\{v,w\}$ to T
  visit(w)

Breadth-First Search

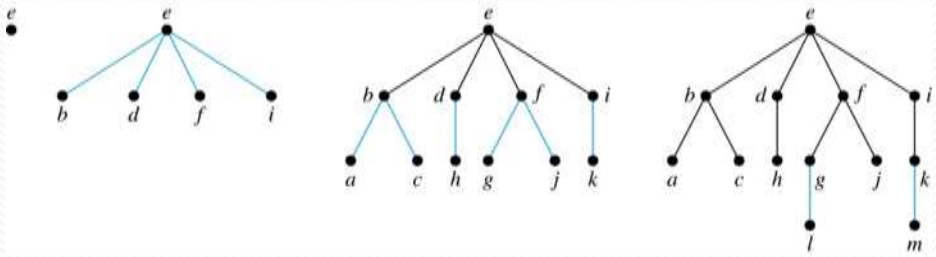
- We can construct a spanning tree using *breadth-first search*. We first arbitrarily choose a root from the vertices of the graph.
  - Then we add all of the edges incident to this vertex and the other endpoint of each of these edges. We say that these are the vertices at level 1.
  - For each vertex added at the previous level, we add each edge incident to this vertex, as long as it does not produce a simple circuit. The new vertices we find are the vertices at the next level.
  - We continue in this manner until all the vertices have been added and we have a spanning tree.

Breadth-First Search (*continued*)

Example: Use breadth-first search to find a spanning tree for this graph.



Solution: We arbitrarily choose vertex *e* as the root. We then add the edges from *e* to *b*, *d*, *f*, and *i*. These four vertices make up level 1 in the tree. Next, we add the edges from *b* to *a* and *c*, the edges from *d* to *h*, the edges from *f* to *j* and *g*, and the edge from *i* to *k*. The endpoints of these edges not at level 1 are at level 2. Next, add edges from these vertices to adjacent vertices not already in the graph. So, we add edges from *g* to *l* and from *k* to *m*. We see that level 3 is made up of the vertices *l* and *m*. This is the last level because there are no new vertices to find.



Breadth-First Search Algorithm

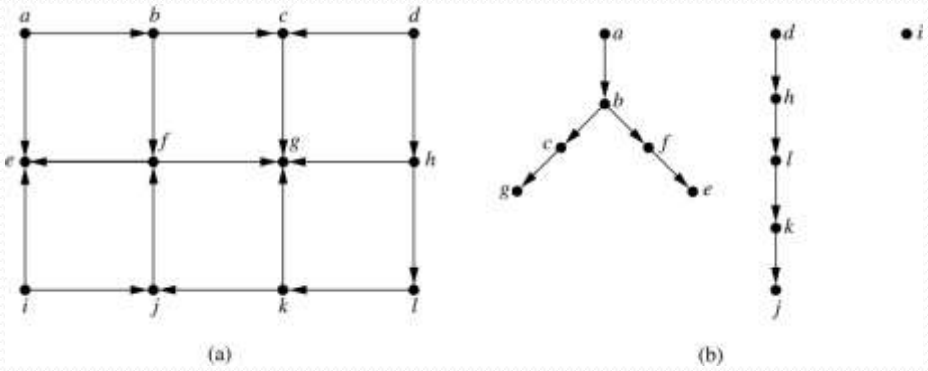
- We now use pseudocode to describe breadth-first search.

procedure *BFS*(*G*: connected graph with vertices $v_1, v_2, \ldots, v_n$)
T := tree consisting only of the vertex v_1
L := empty list
put v_1 in the list L of unprocessed vertices
while L is not empty
    remove the first vertex, v, from L
    for each neighbor w of v
        if w is not in L and not in T **then**
            add w to the end of the list L
            add w and edge $\{v,w\}$ to T

Depth-First Search in Directed Graphs

- Both depth-first search and breadth-first search can be easily modified to run on a directed graph. But the result is not necessarily a spanning tree, but rather a spanning forest.

Example: For the graph in (a), if we begin at vertex *a*, depth-first search adds the path connecting *a*, *b*, *c*, and *g*. At *g*, we are blocked, so we return to *c*. Next, we add the path connecting *f* to *e*. Next, we return to *a* and find that we cannot add a new path. So, we begin another tree with *d* as its root. We find that this new tree consists of the path connecting the vertices *d*, *h*, *l*, *k*, and *j*. Finally, we add a new tree, which only contains *i*, its root.



- To index websites, search engines such as Google systematically explore the web starting at known sites. The programs that do this exploration are known as *Web spiders*. They may use both breath-first search or depth-first search to explore the Web graph.

Minimum Spanning Trees

- A *minimum spanning tree* in a connected weighted graph is a spanning tree that has the smallest possible sum of weights of its edges.
  - Ex: A company plans to build a communications network connecting its five computer centers. Any pair of these centers can be linked with a leased telephone line. Which links should be made to ensure that there is a path between any two computer centers so that the total cost of the network is minimized?

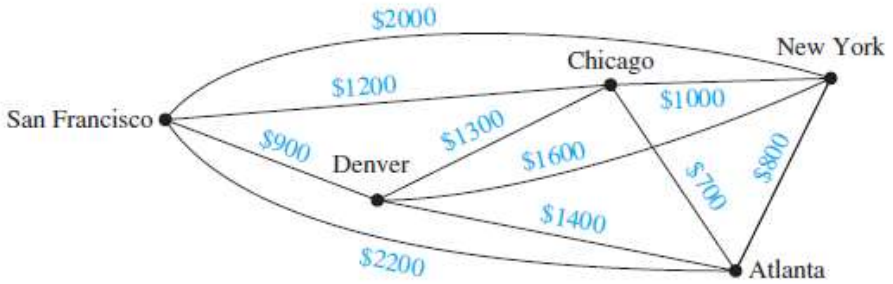


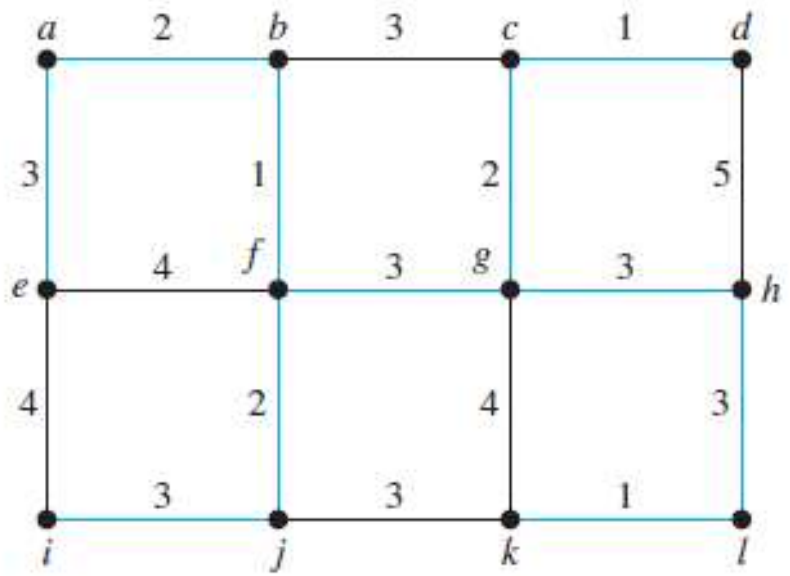
FIGURE 1 **A Weighted Graph Showing Monthly Lease Costs for Lines in a Computer Network.**

Prim's Algorithm

ALGORITHM 1 Prim's Algorithm.

procedure *Prim*(G: weighted connected undirected graph with n vertices)
T := a minimum-weight edge
for $i := 1$ **to** $n - 2$
  e := an edge of minimum weight incident to a vertex in T and not forming a simple circuit in T if added to T
  $T := T$ with e added
return T {T is a minimum spanning tree of G}

Example



(a)

Choice	Edge	Weight
1	{b, f}	1
2	{a, b}	2
3	{f, j}	2
4	{a, e}	3
5	{i, j}	3
6	{f, g}	3
7	{c, g}	2
8	{c, d}	1
9	{g, h}	3
10	{h, l}	3
11	{k, l}	1
	Total:	24

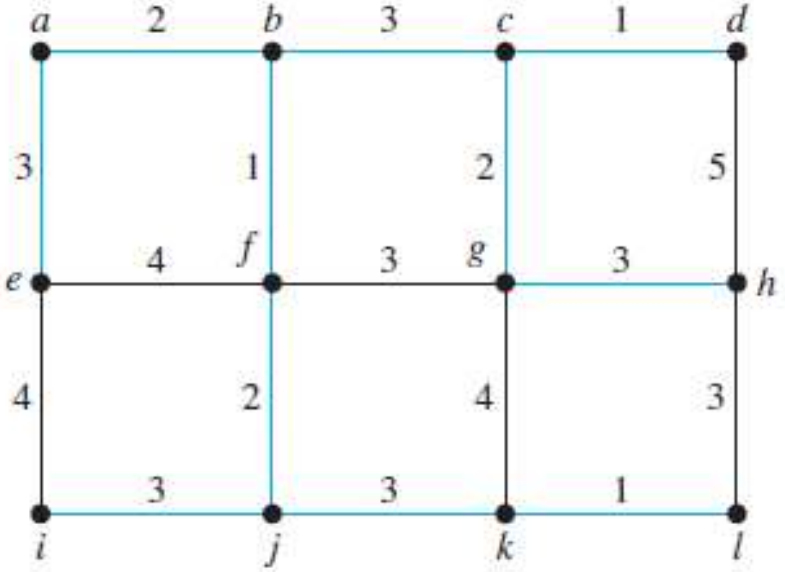
(b)

Kruskal's Algorithm

ALGORITHM 2 Kruskal's Algorithm.

procedure *Kruskal*(G: weighted connected undirected graph with n vertices)
T := empty graph
for $i := 1$ **to** $n - 1$
  e := any edge in G with smallest weight that does not form a simple circuit
    when added to T
  $T := T$ with e added
return T {T is a minimum spanning tree of G}

Example



(a)

Choice	Edge	Weight
1	{c, d}	1
2	{k, l}	1
3	{b, f}	1
4	{c, g}	2
5	{a, b}	2
6	{f, j}	2
7	{b, c}	3
8	{j, k}	3
9	{g, h}	3
10	{i, j}	3
11	{a, e}	3
	Total:	24

(b)

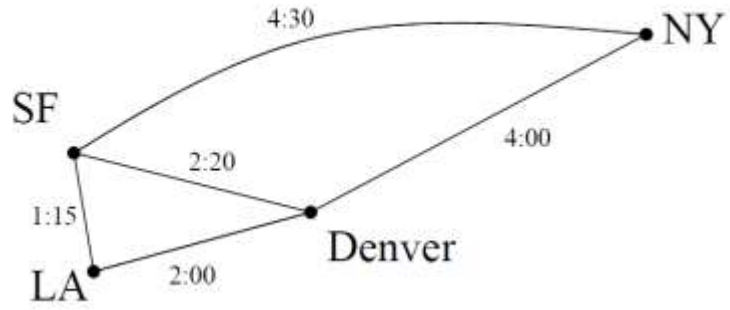
Shortest Path

- Weighted digraph: A directed graph with real valued weights assigned to each edge.
  - G(V,E,w)
- **Length** of a path in a weighted digraph: Sum of the lengths of the edges on the path.
- **Shortest path**: A path between two nodes of least length.

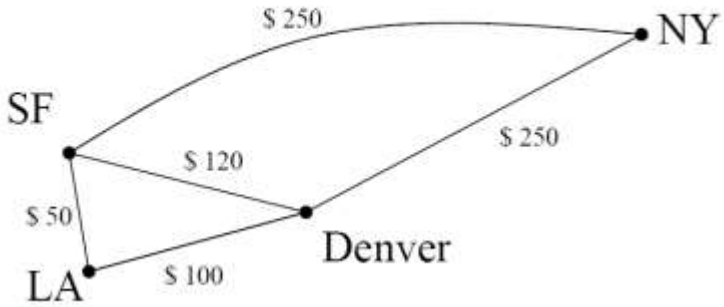
Shortest Path Problems

- Many problems can be modeled using graphs with weights assigned to their edges.

Flight times:



Flight fares



Use of **weighted graphs**:
Problem: Find a path with least possible fare/time/distance.

Shortest Path - Dijkstra's Method

- Let G(V,E) be a weighted digraph all of whose edge weights are positive.
- x and y are vertices of G.

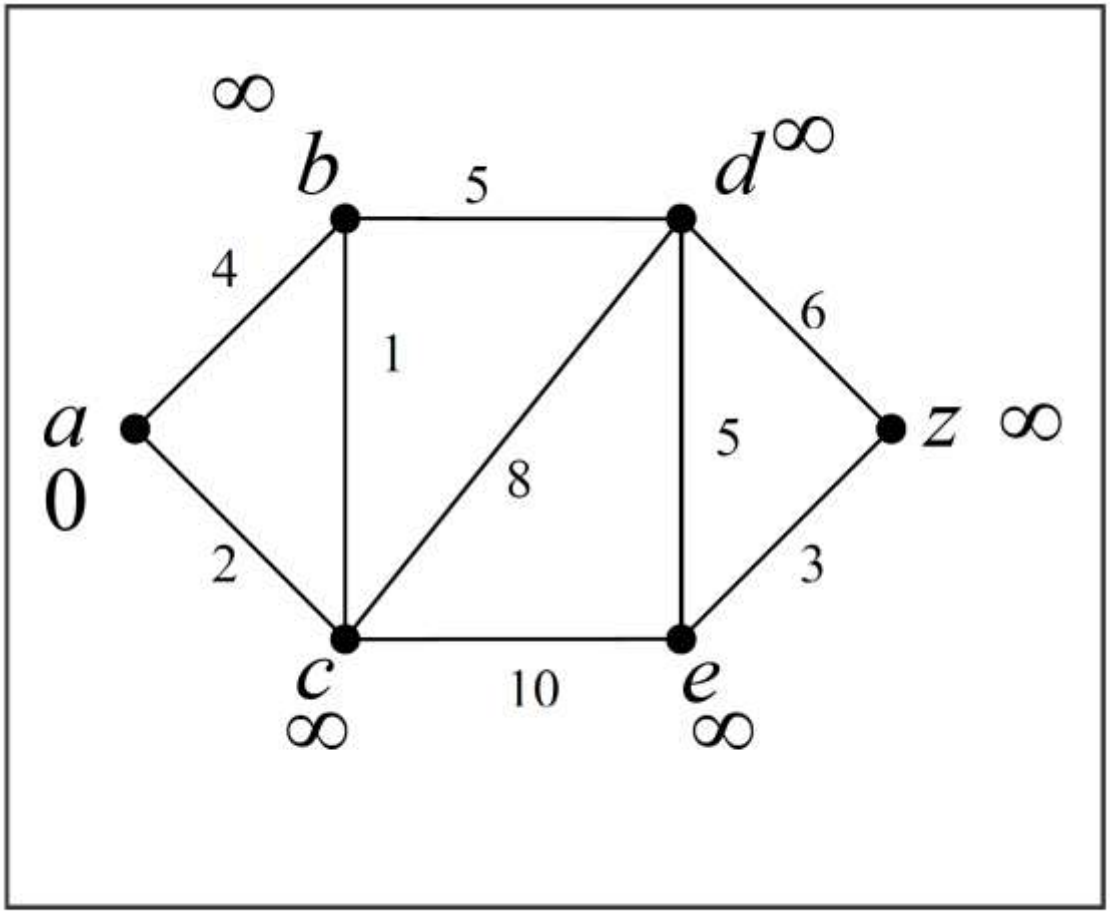
Aim: Find the shortest path from x to y and its length, or show there is none.

- The method uses a search tree technique based on:
  - k^{th} nearest vertex to x is the neighbor of one of the jth nearest vertices to x for some $j < k$.

Dijkstra's Algorithm

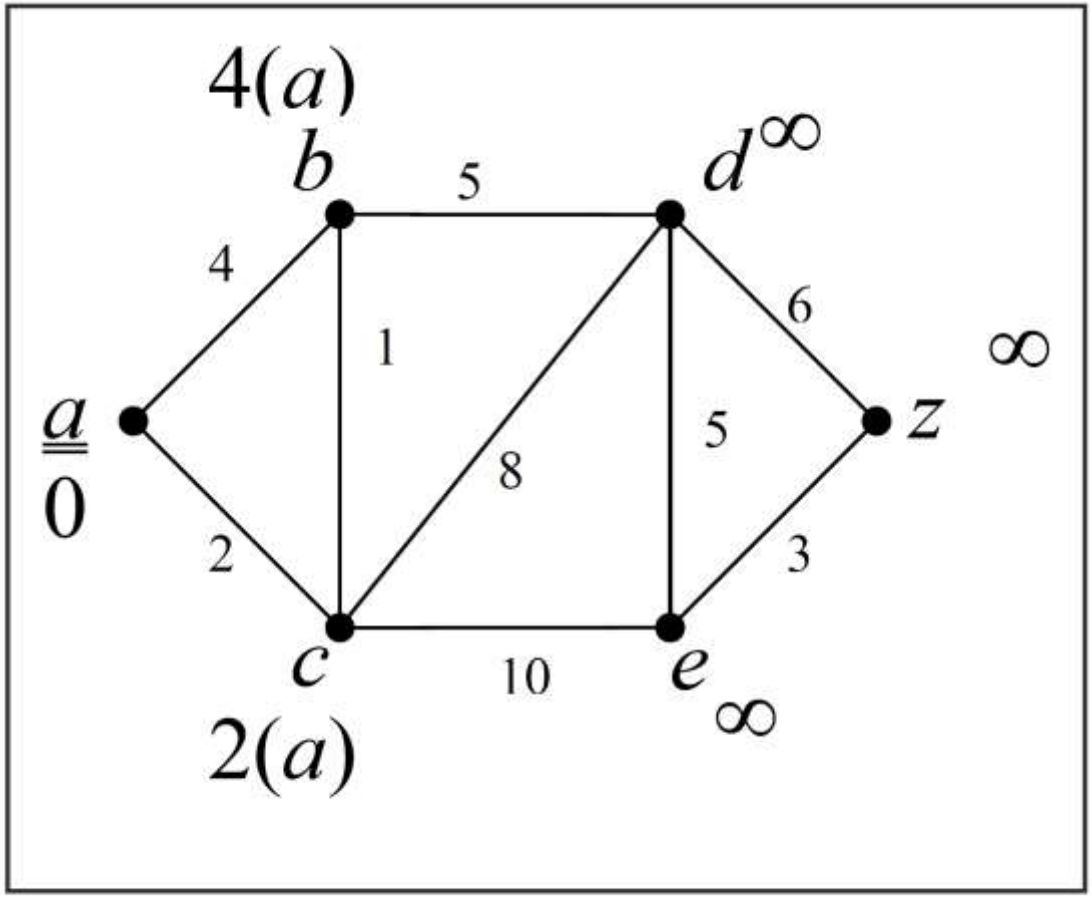
procedure **Dijkstra** (G: positive weighted connected simple graph)
// G has vertices $a = v_0, v_1, \ldots, v_n = z$ and
// weights $w(v_i, v_j)$ where $w(v_i, v_j) = \infty$ if no edge
// initialization
    for ($i = 1$; $i \leq n$; i++) $L(v_i) = \infty$
    $L(a) = 0$
    $S = \varnothing$
    while ($z \notin S$) {
        u = a vertex not in S with $L(u)$ minimal
        $S = S \cup \{u\}$
        for $\forall v \notin S$ {
            if ($L(u) + w(u, v) < L(v)$)
                $L(v) = L(u) + w(u, v)$
        }
    }
$L(z)$ = length of the shortest path from a to z
Dijkstra's algorithm has $O(n^2)$ operations (addition and comparison).

Dijkstra's Algorithm - Example



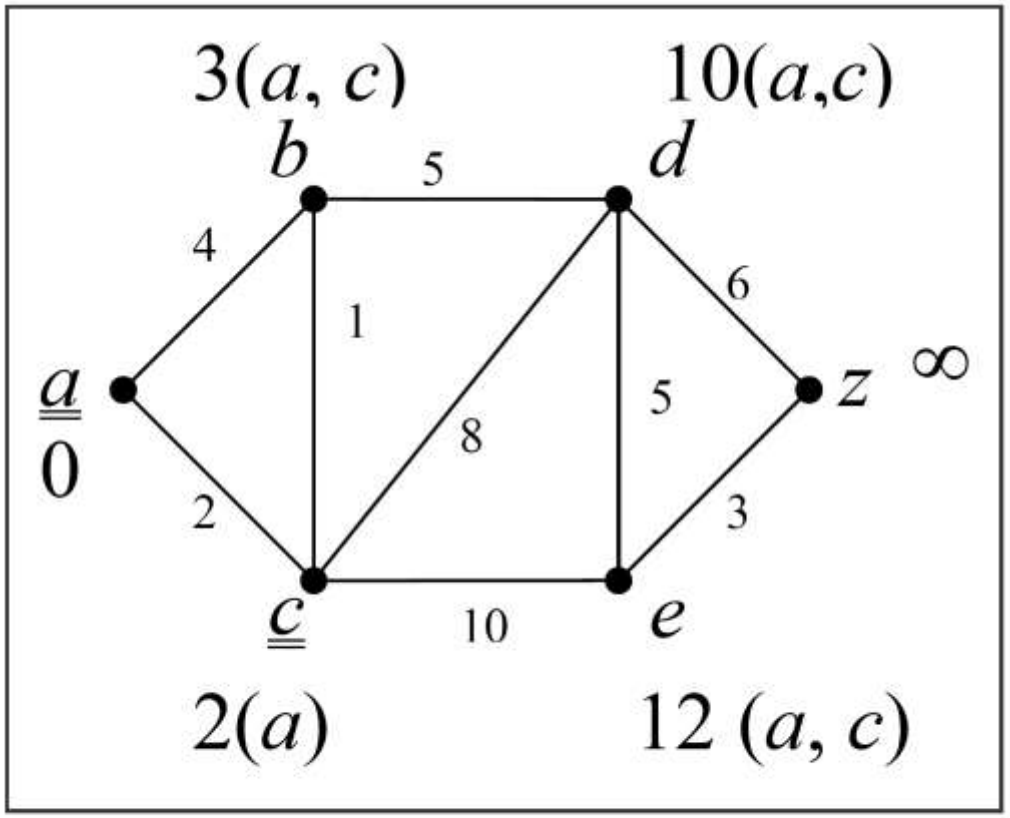
$S=\emptyset$

Dijkstra's Algorithm - Example



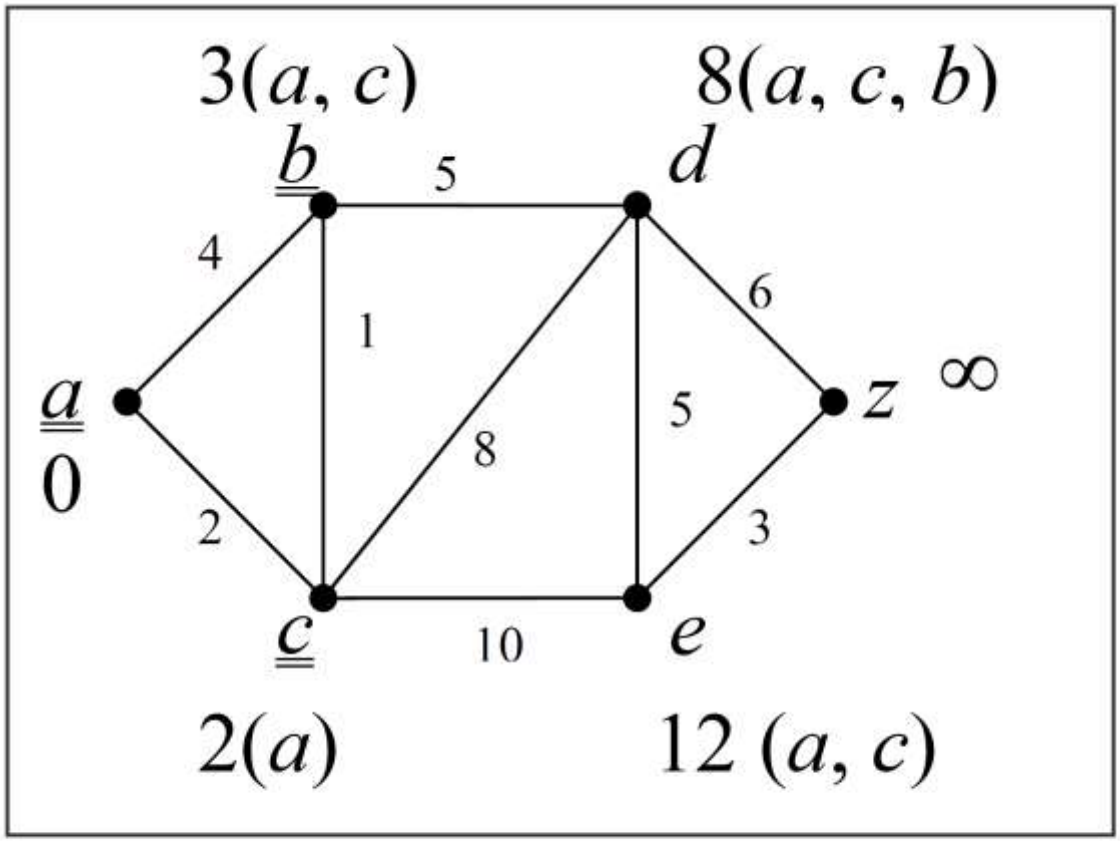
$S=\{a\}$

Dijkstra's Algorithm - Example



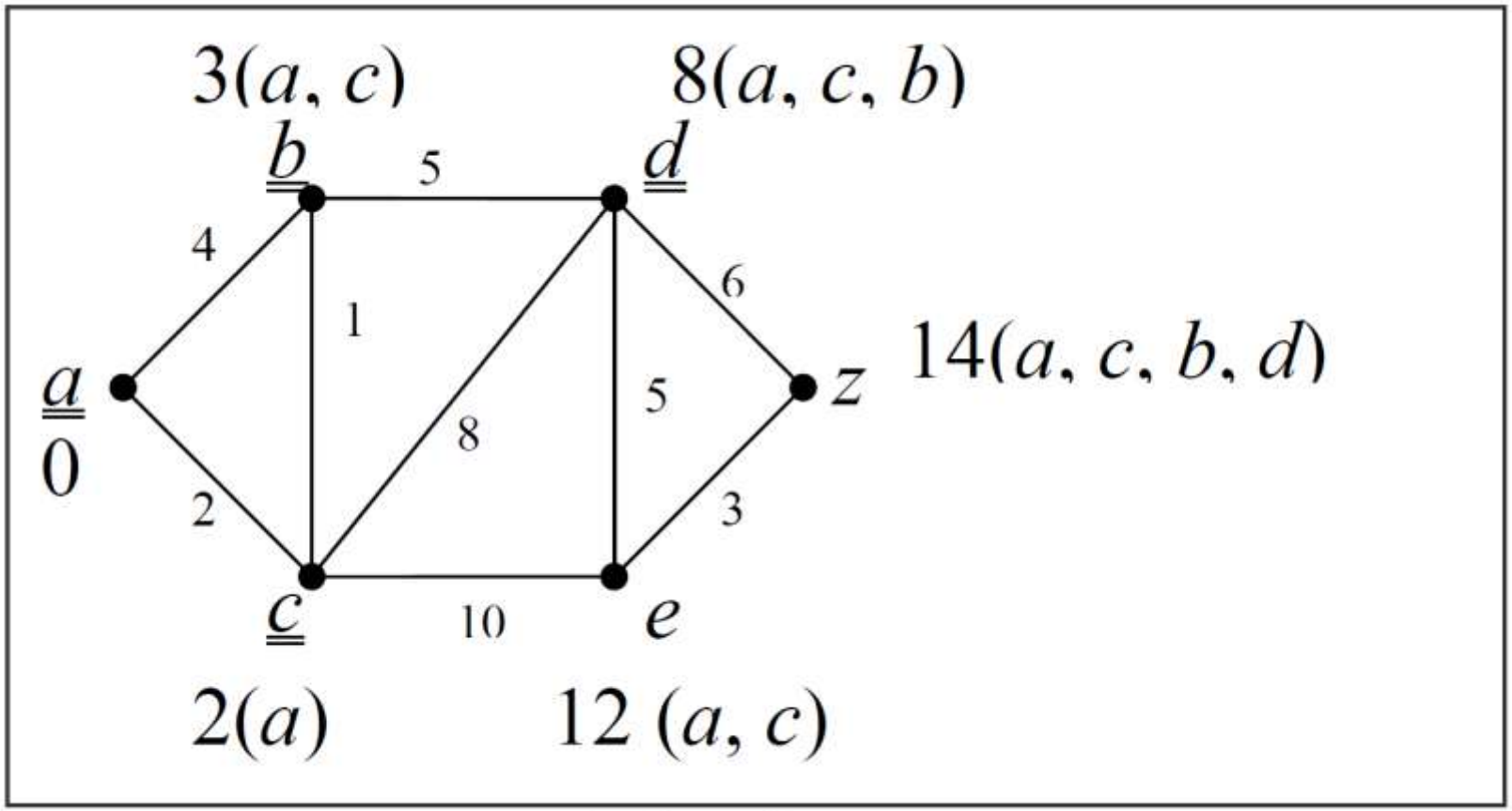
$S=\{a,c\}$

Dijkstra's Algorithm - Example



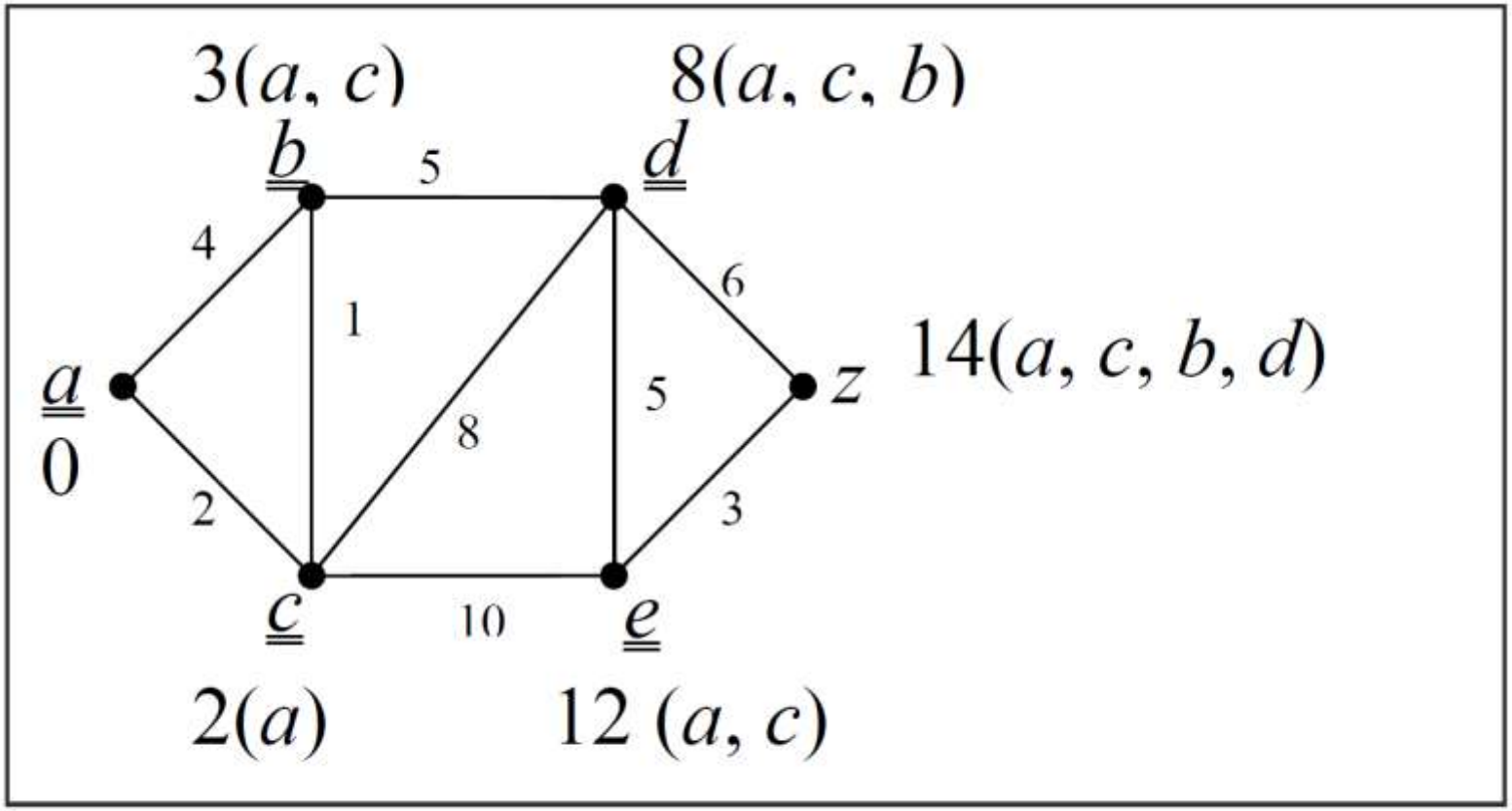
$S=\{a,b,c\}$

Dijkstra's Algorithm - Example



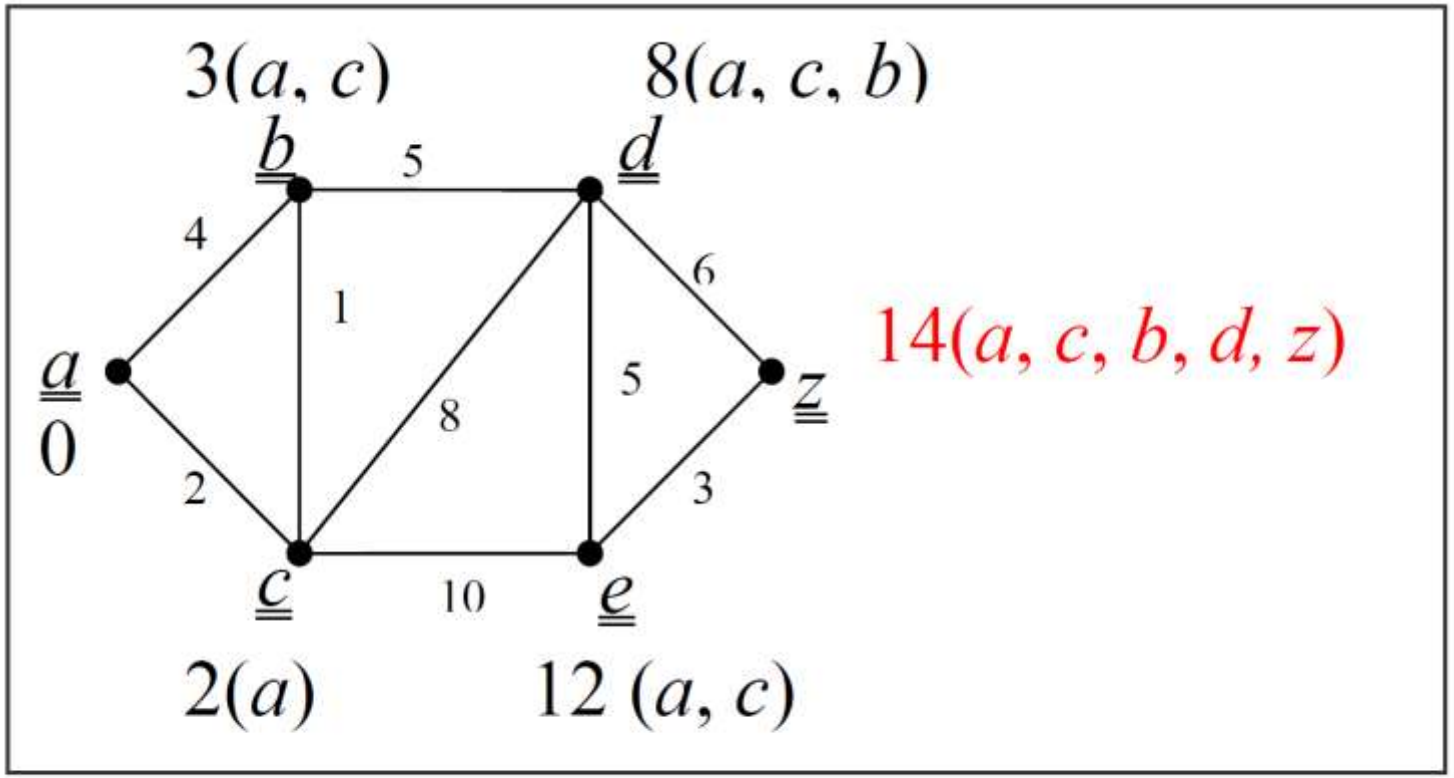
$S=\{a,b,c,d\}$

Diikstra's Algorithm – Example



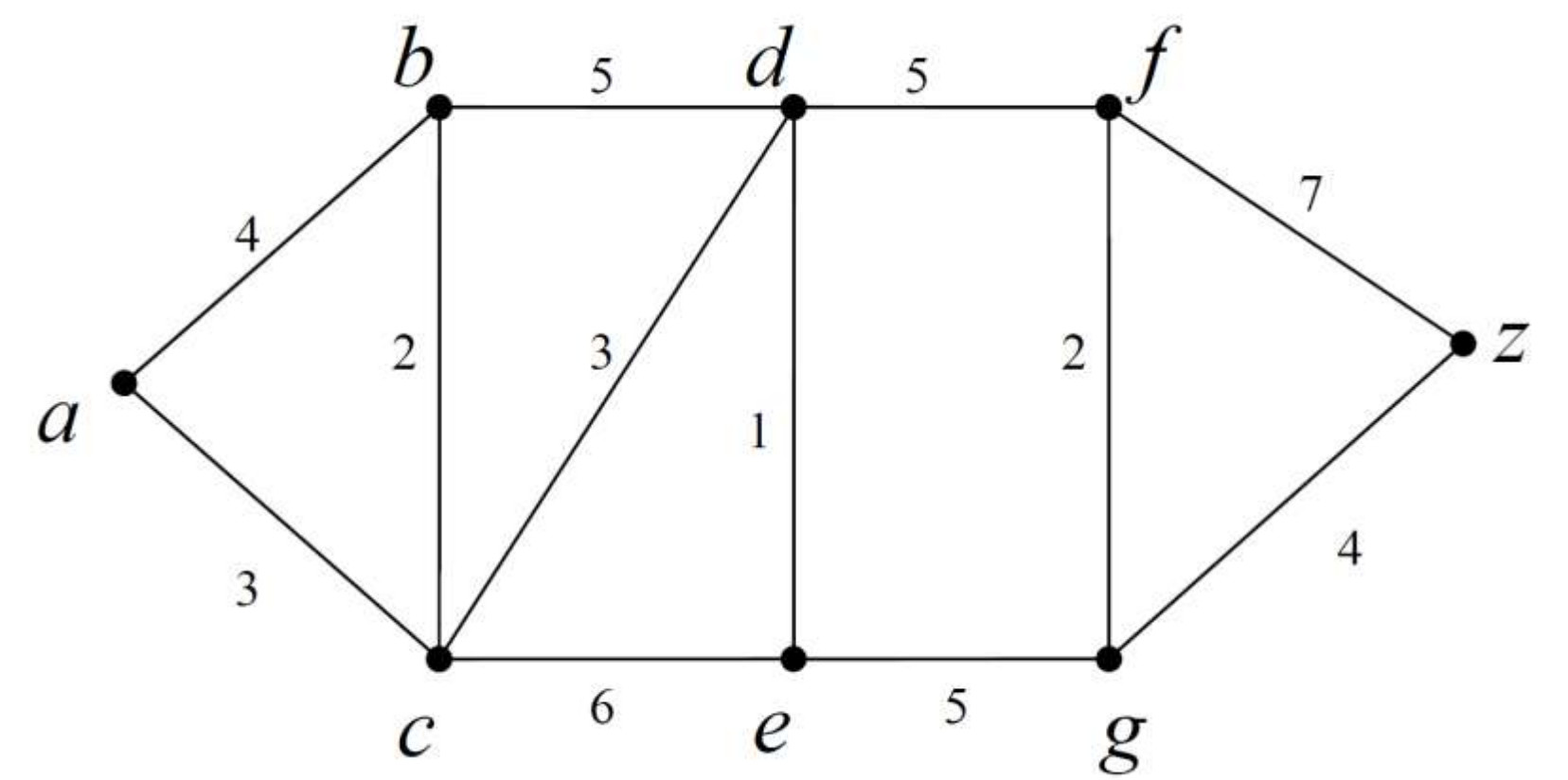
$S=\{a,b,c,d,e\}$

Dijkstra's Algorithm - Example



$S=\{a,b,c,d,e,z\}$

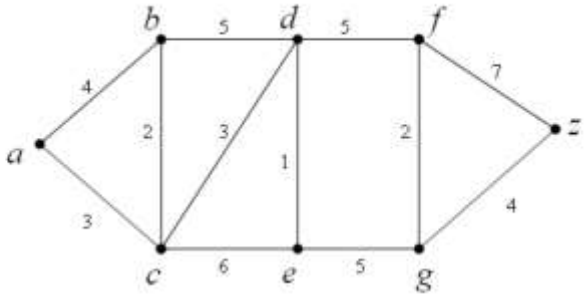
Dijkstra's Algorithm – Example 2



Dijkstra's Algorithm – Example 2

	a	b	c	d	e	f	g	z
	0	∞	∞	∞	∞	∞	∞	∞
$\underline{a}$	0	4 (a)	3 (a)	∞	∞	∞	∞	∞
$\underline{c}$	0	4 (a)	3 (a)	6 (a, c)	9 (a, c)	∞	∞	∞
$\underline{b}$	0	4 (a)	3 (a)	6 (a, c)	9 (a, c)	∞	∞	∞
$\underline{d}$	0	4 (a)	3 (a)	6 (a, c)	7 (a, c, d)	11 (a, c, d)	∞	∞
$\underline{e}$	0	4 (a)	3 (a)	6 (a, c)	7 (a, c, d)	11 (a, c, d)	12 (a, c, d, e)	∞
$\underline{f}$	0	4 (a)	3 (a)	6 (a, c)	7 (a, c, d)	11 (a, c, d)	12 (a, c, d, e)	18 (a, c, d, f)
g	0	4 (a)	3 (a)	6 (a, c)	7 (a, c, d)	11 (a, c, d)	12 (a, c, d, e)	16 (a, c, d, e, g)

$\therefore$ a, c, d, e, g, z is the shortest path.

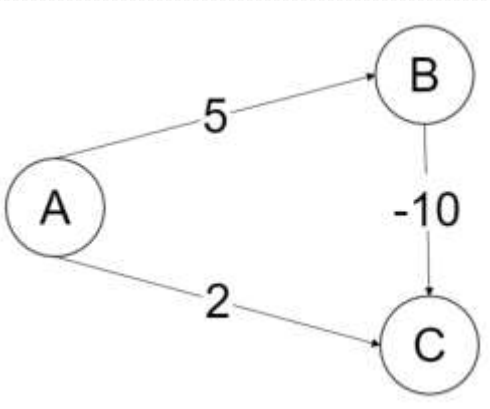


Dijkstra's Algorithm -

- **Theorem**:
  - Dijkstra's algorithm finds the length of a shortest path between two vertices in a connected simple undirected weighted graph.
- **Remark**:
  - Dijkstra's algorithm has several extended versions with complexity $O(n\log n)$.

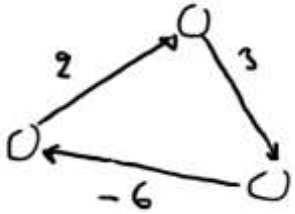
Negative Weights

- Shortest path problem is considered under the assumption that there are no negative cycle in the graph.
- Dijkstra doesn't work for graphs with negative weights.



Negative Cycles

- There are other algorithms that sometimes work for negative weights.
- The crucial question is if a graph contain negative cycles or not.
- Negative cycle: 2+3-6=-1



- The concept of a shortest path is meaningless if there is a negative cycle.
- If a graph does not contain any negative cycles there are algorithms for finding shortest paths.
  - Examples are Bellman-Ford's algorithm and Warshall' algorithm.